

NAME: - K. Sampath Chaitanya

Reg no: - 192572006

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
COMPUTER SCIENCE AND ENGINEERING PROGRAMME

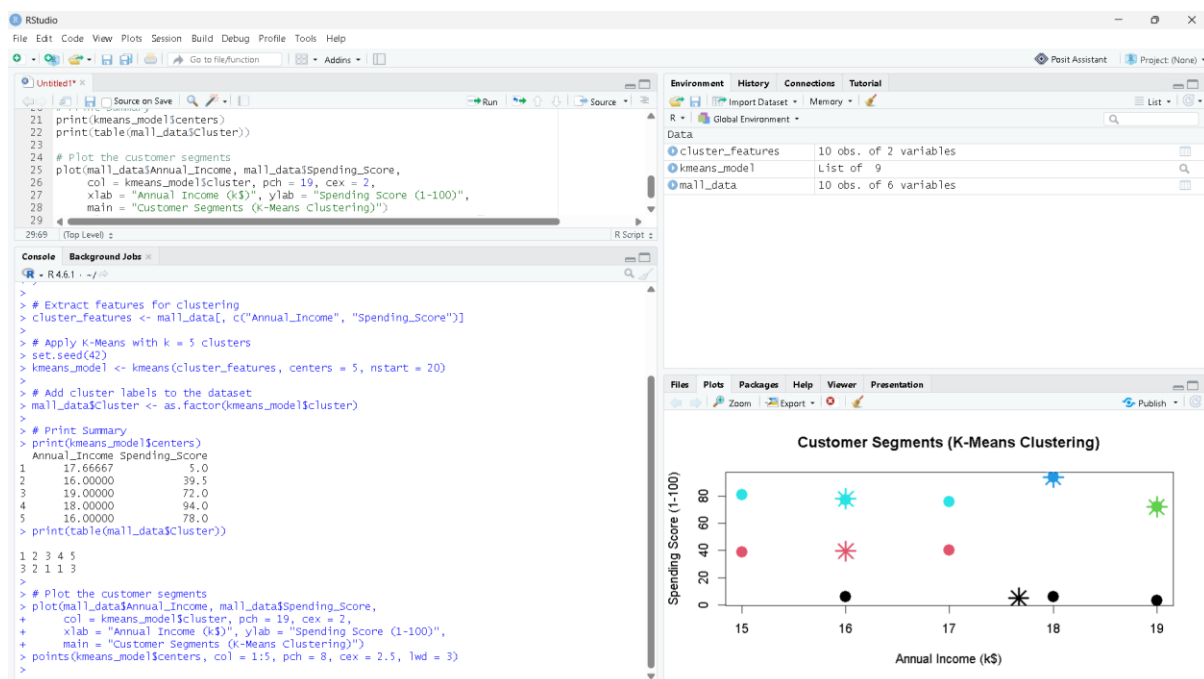
Day 4: List of Programs

Question 1: Customer Segmentation using K-Means Clustering

- **Software / Environment:** R Studio (or WEKA SimpleKMeans)
- **Aim:**

To perform K-Means Clustering on customer supermarket mall data (Annual Income and Spending Score) to segment customers into 5 distinct target groups.

- **Algorithm:**
 1. Create a data frame containing customer features: CustomerID, Gender, Age, Annual_Income, and Spending_Score.
 2. Select the numerical features Annual_Income and Spending_Score for clustering analysis.
 3. Apply the K-Means algorithm using kmeans() with $k = 5$ clusters and nstart = 20.
 4. Label the 5 identified segments (e.g., *Usual Customers*, *Priority Customers*, *Senior Citizen Target Customers*, *Young Target Customers*, *Standard Customers*).
 5. Plot the clusters and display the cluster centers and sizes.



Question 2: K-Means Clustering on Employee Dataset in WEKA

- **Software / Environment: WEKA Explorer (Preprocess & Cluster tabs)**
- **Aim:**

To create a CSV dataset for employee records (EmployeeID, Gender, Age, Salary, Credit), perform K-Means cluster analysis in WEKA, modify the cluster size ($k = 2$ and $k = 3$), and visualize the resulting clusters.

- **Procedure (Steps in WEKA):**
 1. Create a file named `employee_data.csv` using the table values.
 2. Open **WEKA Explorer**, click **Open file...** under the **Preprocess** tab, select file type **CSV data files (*.csv)**, and choose `employee_data.csv`.
 3. Select EmployeeID in the attributes list and click **Remove** so the identifier doesn't affect distance calculations.
 4. Click on the **Cluster** tab at the top.
 5. By default, the clusterer is set to **SimpleKMeans**.
 6. **For $k = 2$:**
 - Click the text box next to **Choose** (where SimpleKMeans is written).
 - Verify numClusters is set to 2.
 - Under *Cluster mode*, choose **Use training set**.
 - Click **OK**, then click **Start**.
 7. **For $k = 3$:**
 - Click the text box next to **Choose** again and change numClusters to 3.
 - Click **OK**, then click **Start**.
 8. **Visualization:** Right-click the result entry in the *Result list* (bottom-left) and select **Visualize cluster assignments** to plot the clusters (e.g., Salary vs Credit or Age).

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Cluster: Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode:
☒ Use training set
☐ Supplied test set Set...
☐ Percentage split % 66
☐ Classes to clusters evaluation (Num) Credit
☒ Store clusters for visualization

Ignore attributes: Start Stop

Result list (right-click for options):
 02:07:00 - SimpleKMeans

Cluster output:
 === Clustering model (full training set) ===
 kMeans
 =====
 Number of iterations: 2
 Within cluster sum of squared errors: 2.9514403292181073
 Initial starting points (random):
 Cluster 0: 444,Female,25,160000,40
 Cluster 1: 111,Male,29,150000,39
 Missing values globally replaced with mean/mode
 Final cluster centroids:

Attribute	Full Data	Cluster#	0	1
	(6.0)		(3.0)	(3.0)
EmployeeID	388.5	444	333	
Gender	Male	Female	Male	Male
Age	27.1667	27	27.3333	
Salary	165000	163333.3333	166666.6667	
Credit	47.3333	48.6667	46	

 Time taken to build model (full training data) : 0 seconds
 === Model and evaluation on training set ===
 Clustered Instances
 0 3 (50%)
 1 3 (50%)

Status: OK Log

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Cluster: Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 3 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode:
☒ Use training set
☐ Supplied test set Set...
☐ Percentage split % 66
☐ Classes to clusters evaluation (Num) Credit
☒ Store clusters for visualization

Ignore attributes: Start Stop

Result list (right-click for options):
 02:07:00 - SimpleKMeans
 02:07:25 - SimpleKMeans

Cluster output:
 kMeans
 =====
 Number of iterations: 3
 Within cluster sum of squared errors: 1.077366255144033
 Initial starting points (random):
 Cluster 0: 444,Female,25,160000,40
 Cluster 1: 111,Male,28,150000,39
 Cluster 2: 222,Male,25,150000,27
 Missing values globally replaced with mean/mode
 Final cluster centroids:

Attribute	Full Data	Cluster#	0	1	2
	(6.0)		(3.0)	(1.0)	(2.0)
EmployeeID	388.5	444	666	166.5	
Gender	Male	Female	Male	Male	Male
Age	27.1667	27	29	26.5	
Salary	165000	163333.3333	200000	150000	
Credit	47.3333	48.6667	72	33	

 Time taken to build model (full training data) : 0 seconds
 === Model and evaluation on training set ===
 Clustered Instances
 0 3 (50%)
 1 1 (17%)
 2 2 (33%)

Status: OK Log

3) Aim

To classify categorical data using the **Naïve Bayes algorithm** in WEKA and compare its performance with **Support Vector Machine (SVM)**. The goal is to evaluate accuracy, precision, recall, and F1-score, and visualize the comparison.

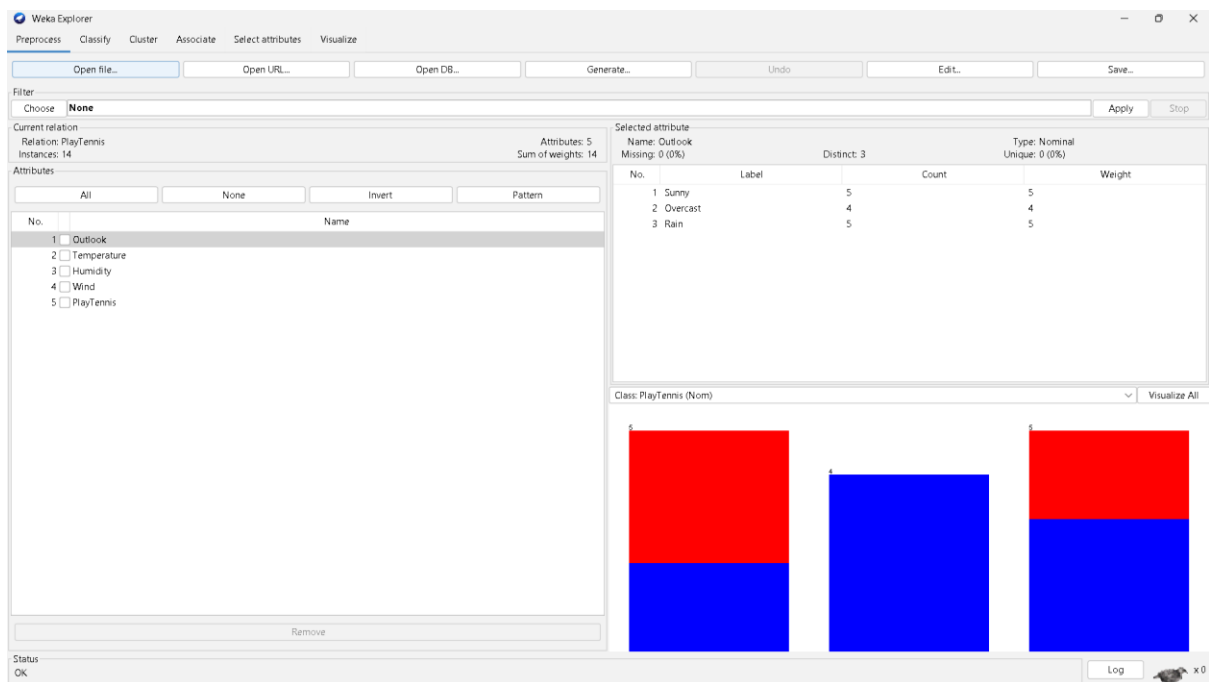
Algorithm

Naïve Bayes

1. Load dataset (categorical attributes + class label).
2. Apply **Naïve Bayes classifier** in WEKA.
3. Perform 10-fold cross-validation.
4. Record accuracy, precision, recall, and F1-score.

Support Vector Machine (SVM)

1. Load the same dataset.
2. Apply **SMO (Sequential Minimal Optimization)** in WEKA (this is WEKA's implementation of SVM).
3. Perform 10-fold cross-validation.
4. Record accuracy, precision, recall, and F1-score.
5. Compare both classifiers using summary results and plot a graph.



Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **NaiveBayes**

Test options:
☐ Use training set
☐ Supplied test set
☒ Cross-validation Folds: 10
☐ Percentage split %: 66
 More options...

(Nom) PlayTennis

Start Stop

Result list (right-click for options):
 17:37:15 - bayes.NaiveBayes

Classifier output:

```

Humidity
High      4.0  5.0
Normal    7.0  2.0
[total]   11.0  7.0

Wind
Weak      7.0  3.0
Strong    4.0  4.0
[total]   11.0  7.0

Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      8      57.1429 %
Incorrectly Classified Instances    6      42.8571 %
Kappa statistic                    -0.0244
Mean absolute error                 0.4374
Root mean squared error             0.4916
Relative absolute error             51.8431 %
Root relative squared error         99.6492 %
Total Number of Instances          14

=== Detailed Accuracy By Class ===

          TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
          -----  -
0.778   0.800   0.636   0.778   0.700     -0.026   0.578   0.697   Yes
0.200   0.222   0.393   0.200   0.250     -0.026   0.578   0.557   No
Weighted Avg.   0.571   0.594   0.528   0.571   0.539     -0.026   0.578   0.647

=== Confusion Matrix ===
a b <-- classified as
7 2 | a = Yes
4 1 | b = No
  
```

Status: OK

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **SMO-C 1.0-L 0.001-P 1.0E-12-N 0-V -1-W 1-K "weka.classifiers.functions.supportVector.PolyKernel-E 1.0-C 25000"-calibrator "weka.classifiers.functions.Logistic-R 1.0E-8-M -1-num-decimal-places 4"**

Test options:
☐ Use training set
☐ Supplied test set
☒ Cross-validation Folds: 10
☐ Percentage split %: 66
 More options...

(Nom) PlayTennis

Start Stop

Result list (right-click for options):
 17:37:15 - bayes.NaiveBayes
 17:39:33 - functions.SMO

Classifier output:

```

+ 0.4799 * (normalized) Outlook=Rain
+ 0.3596 * (normalized) Temperature=Hot
+ -0.3194 * (normalized) Temperature=Mild
+ -0.0402 * (normalized) Temperature=Cool
+ -1.0402 * (normalized) Humidity=Normal
+ 0.6806 * (normalized) Wind=Strong
- 0.4004

Number of kernel evaluations: 80 (84.82% cached)

Time taken to build model: 0.01 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      8      57.1429 %
Incorrectly Classified Instances    6      42.8571 %
Kappa statistic                    -0.0244
Mean absolute error                 0.4286
Root mean squared error             0.6547
Relative absolute error             90 %
Root relative squared error         132.6919 %
Total Number of Instances          14

=== Detailed Accuracy By Class ===

          TP Rate  FP Rate  Precision  Recall  F-Measure  MCC      ROC Area  PRC Area  Class
          -----  -
0.778   0.800   0.636   0.778   0.700     -0.026   0.489   0.638   Yes
0.200   0.222   0.393   0.200   0.250     -0.026   0.489   0.352   No
Weighted Avg.   0.571   0.594   0.528   0.571   0.539     -0.026   0.489   0.536

=== Confusion Matrix ===
a b <-- classified as
7 2 | a = Yes
4 1 | b = No
  
```

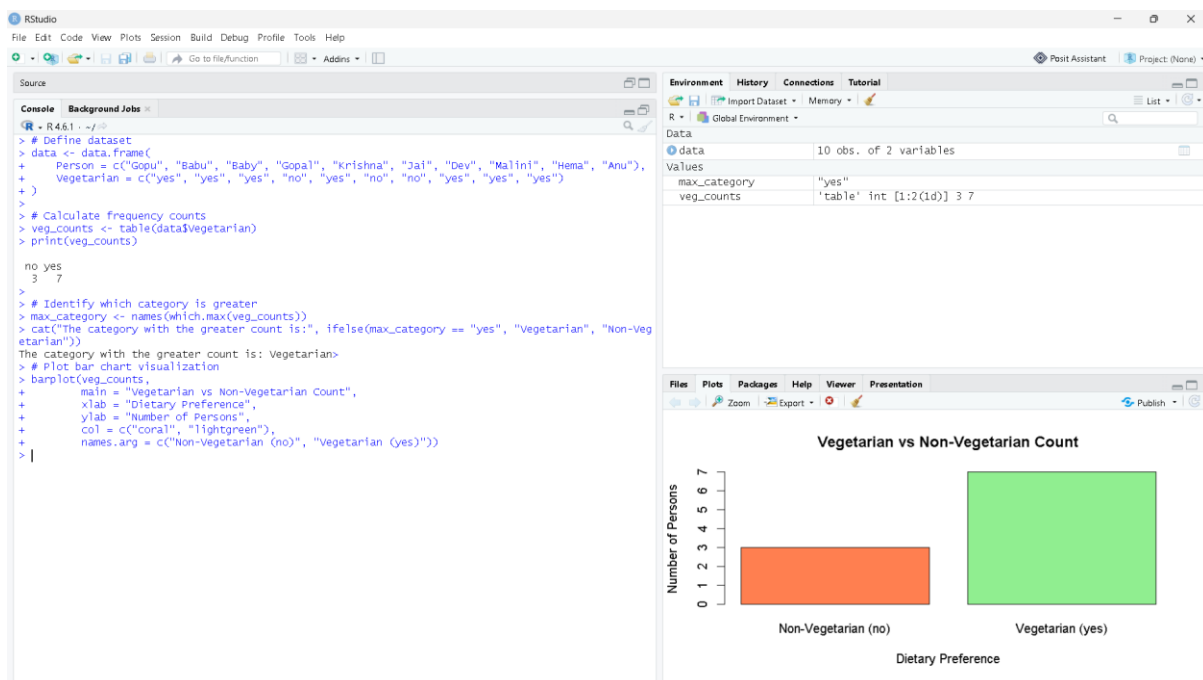
Status: OK

4.Aim

To determine the frequency count of vegetarian and non-vegetarian individuals from the provided dataset and identify which category has the greater count.

Algorithm

1. **Initialize Data:** Load or define the list of individuals along with their respective dietary preferences (yes for vegetarian, no for non-vegetarian).
2. **Initialize Counters:** Set `count_vegetarian = 0` and `count_non_vegetarian = 0`.
3. **Iterate and Count:** For each record in the dataset:
 - If `Vegetarian == "yes"`, increment `count_vegetarian` by 1.
 - If `Vegetarian == "no"`, increment `count_non_vegetarian` by 1.
4. **Compare Values:** Check whether `count_vegetarian > count_non_vegetarian`.
5. **Display Results:** Output the individual counts and state which category has the greater value.

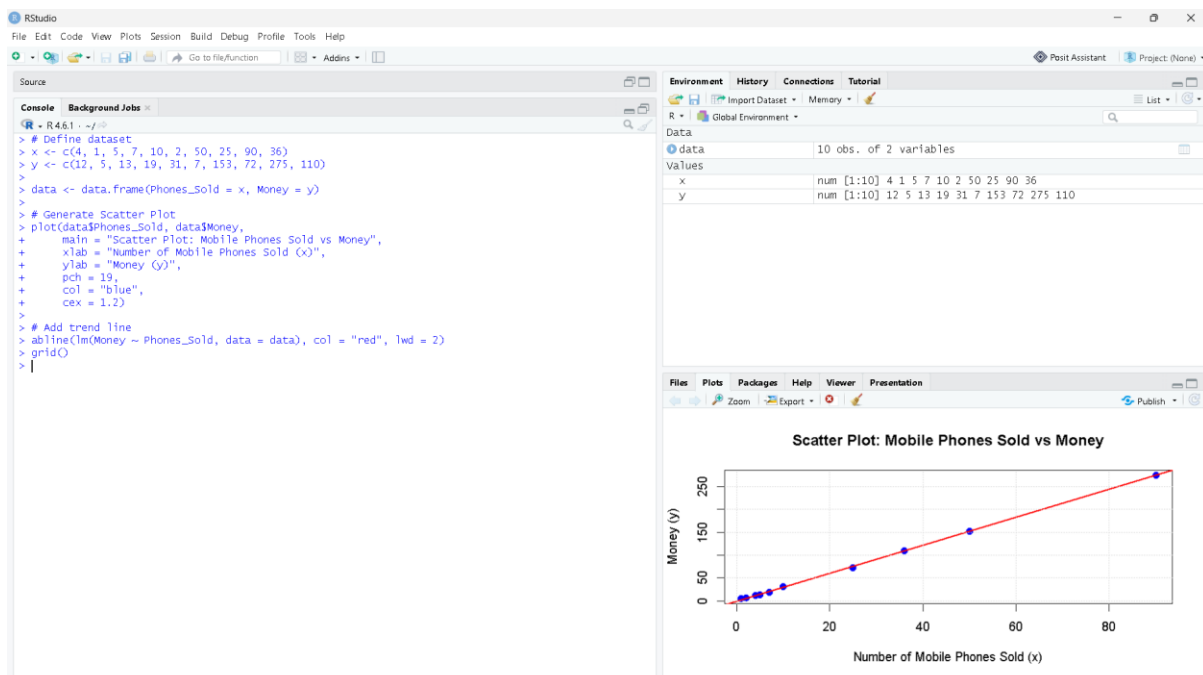


5.Aim

To create a scatter plot visualizing the relationship between the number of mobile phones sold (x) and the money earned (y).

Algorithm

1. **Initialize Data:** Define the numeric vector x representing the number of mobile phones sold and vector y representing the corresponding money/revenue.
2. **Combine Vectors:** Create a dataframe containing variables x and y .
3. **Generate Scatter Plot:** Plot y against x using Cartesian coordinates where points are placed at (x_i, y_i) .
4. **Configure Graph Properties:** Add labels for the X-axis ("Number of Mobile Phones Sold"), Y-axis ("Money Earned"), and a chart title.
5. **Display and Analyze:** Render the plot and observe the linear correlation between the units sold and the total revenue.



6.Aim

To implement and generate association rules using the FP-Growth algorithm in WEKA on the given transaction dataset with a minimum support of 50% and a minimum confidence of 75%.

Algorithm (WEKA Procedure)

1. **Prepare Dataset:** Create an ARFF file with binary attributes representing each unique item in the transactions (t for present, ? for absent).
2. **Launch WEKA Explorer:** Open the WEKA GUI Chooser and select **Explorer**.
3. **Load Data:**
 - Navigate to the **Preprocess** tab.
 - Click **Open file...** and load the prepared .arff file.
4. **Select Associator:**
 - Go to the **Associate** tab.
 - Click **Choose** and select weka.associations.FPGrowth.
5. **Configure Parameters:**
 - Click on the text next to the Choose button to open the properties window.
 - Set lowerBoundMinSupport to **0.5** (50% support).
 - Set minMetric (confidence) to **0.75** (75% confidence).
 - Set metricType to **Confidence** (default).
 - Set findAllRulesForSupportLevel to **True** (or set numRulesToFind to **10**).
 - Click **OK**.
6. **Execute and Record:**
 - Click the **Start** button to run the algorithm.
 - View and record the frequent patterns and generated association rules from the Classifier Output window.

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Open file... Open URL... Open DB... Generate... Undo Edit... Save...

Filter: Choose **None** Apply Stop

Current relation: Relation: grocery_transactions
Instances: 5

Attributes: 6 Sum of weights: 5

All None Invert Pattern

No.	Name
1	☒ Bread
2	☐ Cheese
3	☐ Egg
4	☐ Juice
5	☐ Milk
6	☐ Yogurt

Remove

Status: OK

Selected attribute: Name: Bread
Missing: 1 (20%)

No.	Label	Distinct	Count	Type: Nominal Unique: 0 (0%)	Weight
1	t	4	4		

Class: Yogurt (Nom) Visualize All

Log x 0

Weka Explorer

Preprocess Classify Cluster **Associate** Select attributes Visualize

Associator: Choose **FPGrowth** -P 2 -I -1 -N 10 -T 0 -C 0.75 -D 0.05 -U 1.0 -M 0.5

Start Stop

Result list (right-click for...): 175241 - FPGrowth

==== Run information ====

```

Scheme: weka.associations.FPGrowth -P 2 -I -1 -N 10 -T 0 -C 0.75 -D 0.05 -U 1.0 -M 0.5
Relation: grocery_transactions
Instances: 5
Attributes: 6
  Bread
  Cheese
  Egg
  Juice
  Milk
  Yogurt
==== Associator model (full training set) ====

FPGrowth found 4 rules (displaying top 4)
1. [Cheese=t]: 3 ==> [Juice=t]: 3 <conf: (1)> lift: (1.25) lev: (0.12) conv: (0.6)
2. [Juice=t]: 4 ==> [Bread=t]: 3 <conf: (0.75)> lift: (0.94) lev: (-0.04) conv: (0.4)
3. [Bread=t]: 4 ==> [Juice=t]: 3 <conf: (0.75)> lift: (0.94) lev: (-0.04) conv: (0.4)
4. [Juice=t]: 4 ==> [Cheese=t]: 3 <conf: (0.75)> lift: (1.25) lev: (0.12) conv: (0.8)

```

Status: OK

Log x 0

7.Aim

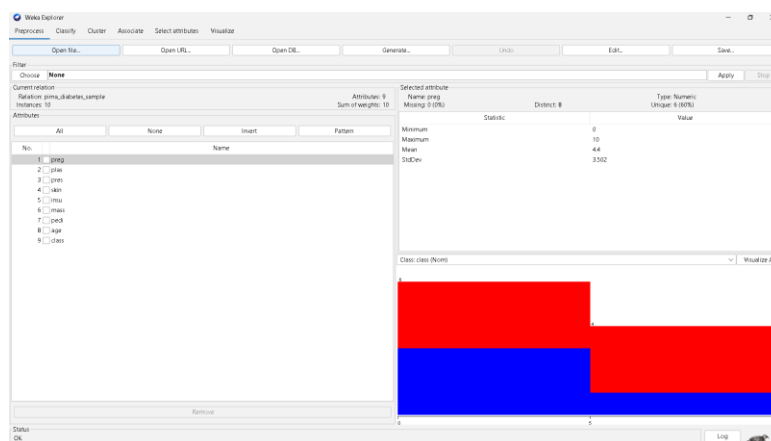
To predict diabetes using the Decision Tree (J48) classifier in WEKA, compare its performance with the Support Vector Machine (SMO/SVM) classifier in terms of Accuracy and F1-Measure, and visualize the performance comparison.

Algorithm (WEKA & Analysis Procedure)

1. **Load Dataset:** Open the WEKA Explorer, go to the **Preprocess** tab, and load the diabetes dataset (diabetes.arff / Pima Indians Diabetes dataset).
2. **Train Decision Tree Classifier (J48):**
 - Navigate to the **Classify** tab.
 - Click **Choose** → trees → **J48**.
 - Select **Cross-validation** (Folds = 10).
 - Click **Start** and record the Correctly Classified Instances (Accuracy), Precision, Recall, and F-Measure.
3. **Train Support Vector Machine Classifier (SMO):**
 - Under the **Classify** tab, click **Choose** → functions → **SMO**.
 - Keep **Cross-validation** (Folds = 10).
 - Click **Start** and record the performance metrics.
4. **Calculate F1-Measure Formula:**

$$\text{F1-Measure} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. **Compare & Visualize:** Compare both models side-by-side using a comparison table and generate a bar chart in RStudio to illustrate the differences in Accuracy and F1-Score.



Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **J48 -C 0.25 -M 2**

Test options:
☐ Use training set
☐ Supplied test set
☒ Cross-validation Folds: **10**
☐ Percentage split %: **66**
 More options...

(Nom) class: **Start** **Stop**

Result list (right-click for options):
 1755:53 - trees.J48

Classifier output:

```

J48 pruned tree
-----
plea <= 116: tested_negative (5.0/1.0)
plea > 116: tested_positive (5.0)

Number of Leaves :    2
Size of the tree :    3

Time taken to build model: 0.01 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      6      60 %
Incorrectly Classified Instances    4      40 %
Kappa statistic                    0.2308
Mean absolute error                 0.435
Root mean squared error             0.5889
Relative absolute error             82.5 %
Root relative squared error        110.1192 %
Total Number of Instances          10

=== Detailed Accuracy By Class ===

          TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
          -----  -
          0.750    0.500    0.500    0.750    0.600      0.250    0.438    0.475    tested_negative
          0.500    0.250    0.750    0.500    0.600      0.250    0.438    0.660    tested_positive
Weighted Avg.   0.600    0.350    0.650    0.600    0.600      0.250    0.438    0.586

=== Confusion Matrix ===
a b   <-- classified as
3 1 | a = tested_negative
3 3 | b = tested_positive

```

Status: OK Log

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Choose **SMO -C 1.0 -L 0.001 -P 1.0E-12 -N 0 -V -I -W 1 -K "weka.classifiers.functions.supportVector.PolyKernel -E 1.0 -C 250007" -calibrator "weka.classifiers.functions.Logistic -R 1.0E-8 -M -1 -num-decimal-places 4"**

Test options:
☐ Use training set
☐ Supplied test set
☒ Cross-validation Folds: **10**
☐ Percentage split %: **66**
 More options...

(Nom) class: **Start** **Stop**

Result list (right-click for options):
 1755:53 - trees.J48
 1757:09 - functions.SMO

Classifier output:

```

+ 0.4631 * (normalized) plea
+ 0.3333 * (normalized) skin
+ 0.2921 * (normalized) insu
+ -0.4268 * (normalized) mass
+ 1.1853 * (normalized) pedi
+ 1.0408 * (normalized) age
- 1.173

Number of kernel evaluations: 47 (62.097% cached)

Time taken to build model: 0.01 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      3      30 %
Incorrectly Classified Instances    7      70 %
Kappa statistic                   -0.5217
Mean absolute error               0.7
Root mean squared error           0.8367
Relative absolute error           132.7586 %
Root relative squared error       156.4601 %
Total Number of Instances          10

=== Detailed Accuracy By Class ===

          TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
          -----  -
          0.000    0.500    0.000    0.000    0.000     -0.535    0.250    0.400    tested_negative
          0.500    1.000    0.429    0.500    0.462     -0.535    0.250    0.514    tested_positive
Weighted Avg.   0.300    0.800    0.257    0.300    0.277     -0.535    0.250    0.469

=== Confusion Matrix ===
a b   <-- classified as
0 4 | a = tested_negative
3 3 | b = tested_positive

```

Status: OK Log

8.Aim

To implement data discretization and binning in R by partitioning a student's model exam marks into three bins using equal-frequency (equi-depth), equal-width, and clustering (k-means) methods, and to plot their histograms.

Algorithm

1. **Load and Sort Data:** Initialize the dataset of exam marks and arrange the 17 numbers in ascending order:

50,55,55,55,58,59,60,61,63,63,65,65,67,71,71,72,75

2. **Method 1 - Equal-Frequency (Equi-Depth) Partitioning:**

- Calculate bin size: $N/k = 17/3 \approx 5.67$ values per bin.
- Split the sorted array into 3 groups containing approximately equal numbers of elements (e.g., Bin 1: 6 values, Bin 2: 6 values, Bin 3: 5 values).

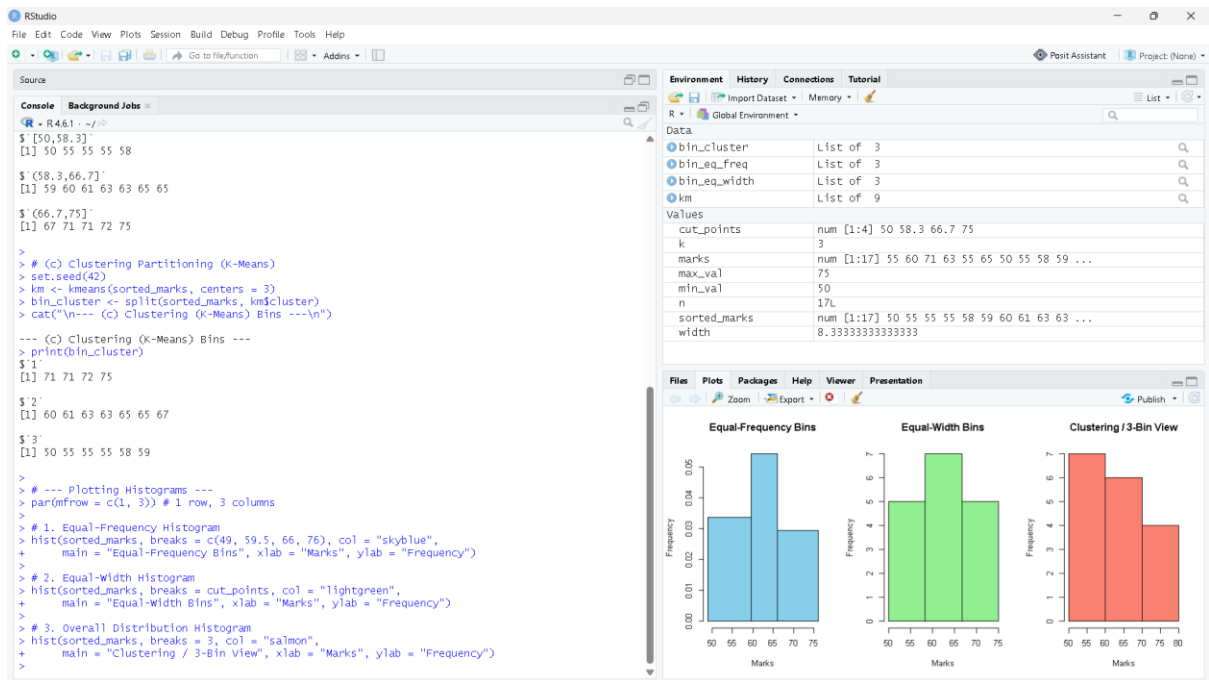
3. **Method 2 - Equal-Width Partitioning:**

- Compute range width: $W = \frac{\text{Max}-\text{Min}}{k} = \frac{75-50}{3} = 8.33$.
- Define bin intervals: [50,58.33), [58.33,66.67), and [66.67,75].
- Assign values into each interval.

4. **Method 3 - Clustering Partitioning (K-Means):**

- Apply 1D k-means clustering with $k = 3$.
- Assign points to the cluster with the nearest mean centroid.

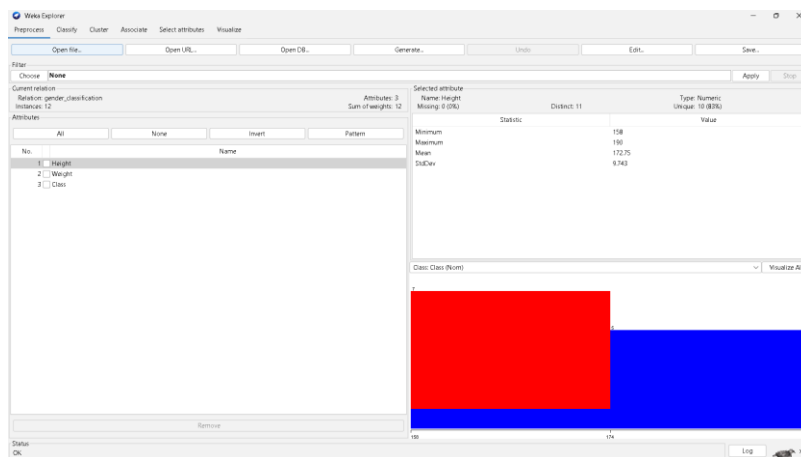
5. **Plot Histograms:** Generate histograms in RStudio to visually represent the partitioned bins.



9.Aim

To create an ARFF dataset representing the given Decision Tree based on Height and Weight, extract classification rules via rule-based induction, evaluate classification accuracy, and compare the decision tree and rule-based models using a confusion matrix.

- **Algorithm**
- **Dataset Synthesis:** Create instances covering all leaf nodes of the given decision tree:
- Branch 1: Height > 180 $\Rightarrow$ Male
- Branch 2: Height $\leq$ 180 AND Weight > 80 $\Rightarrow$ Male
- Branch 3: Height $\leq$ 180 AND Weight $\leq$ 80 $\Rightarrow$ Female
- **ARFF Construction:** Define attributes @attribute Height numeric, @attribute Weight numeric, and @attribute Class {Male, Female}.
- **Rule Induction (RIPPER / JRip):** Traverse decision tree paths from root to leaves to extract IF-THEN rules.
- **Classification & Confusion Matrix in WEKA:**
- Load the ARFF file into WEKA.
- Run the **J48 Decision Tree** under the Classify tab.
- Run the **JRip (Rule-based classifier)** under the Classify tab.
- Generate and compare the confusion matrices and accuracy scores.



Weka Explorer

PreprocessClassifyClusterAssociateSelect attributesVisualize

Classifier

ChooseJ48C.0.25-M 2

Test options

☐ Use training set

☐ Supplied test set

☒ Cross-validation

☐ Percentage split

Folds10

h50

More options...

(Nom) Class

Start

Stop

Result list (right-click for options)

Result

Classifier output

J48 pruned tree

Height = 172; Female (7.0/1.0)
Height > 172; Male (5.0)
Number of leaves = 2
Size of the tree = 3
Time taken to build model: 0 seconds
==== Detailed cross-validation results ====
==== Summary ====
Correctly Classified Instances975
Incorrectly Classified Instances325
Kappa statistic0.5
Mean absolute error0.3124
Root mean squared error0.5114
Relative absolute error60.7704 %
Root relative squared error97.2411 %
Total Number of Instances12
==== Detailed Accuracy By Class ====
TP RateFP RatePrecisionRecallF-MeasureMCCROC AreaPRC AreaClass
0.6670.1670.8000.6670.7270.5070.6110.700Male
0.8330.3330.7140.8330.7690.5070.6110.679Female
Weighted Avg.0.7500.2500.7570.7500.7480.5070.6110.689
==== Confusion Matrix ====
a b <-- classified as
a 2 | a = Male
2 1 | b = Female

Status

OK

Log

x0

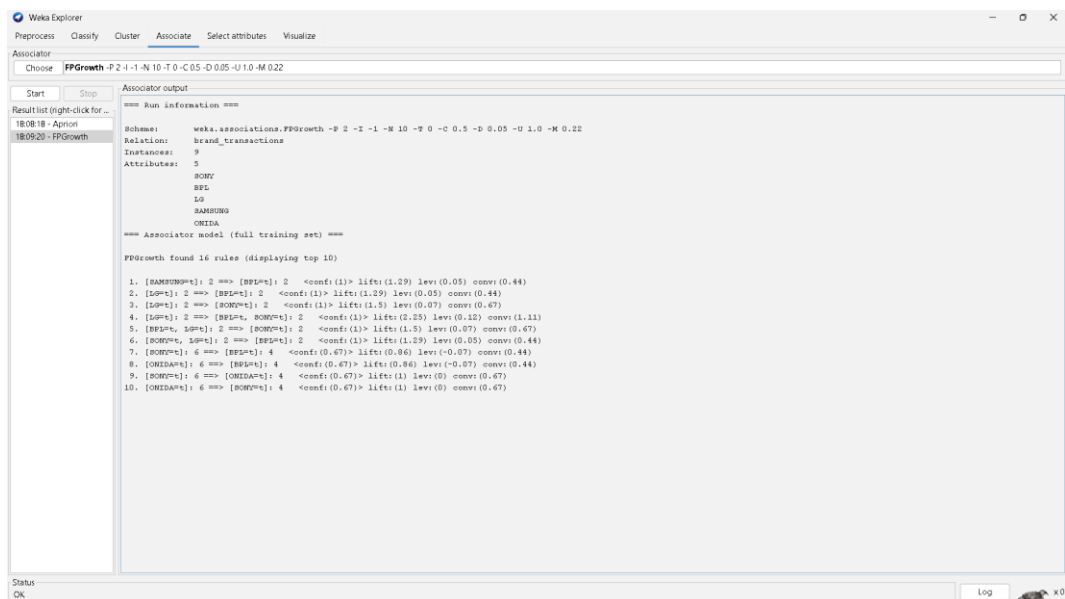
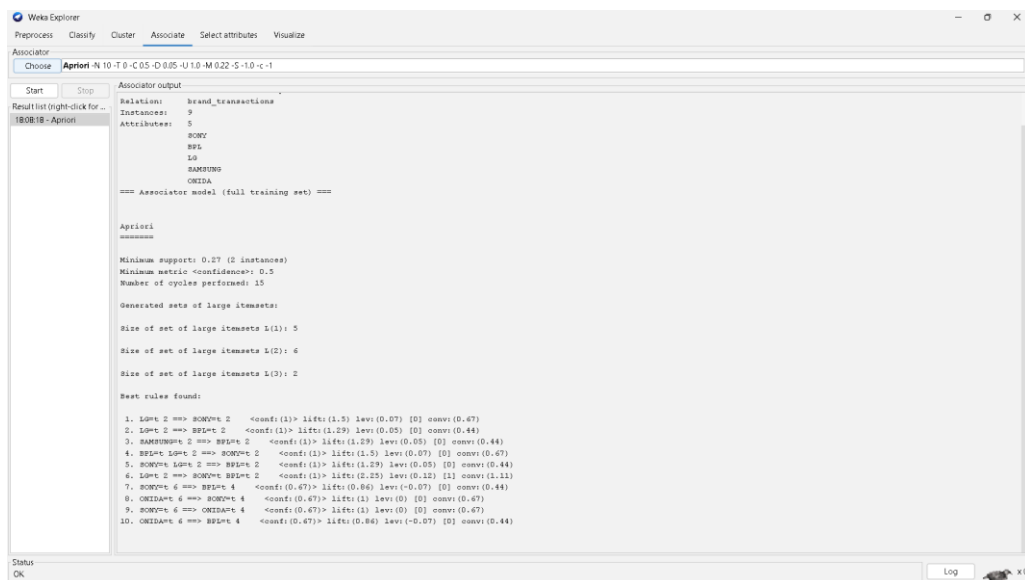
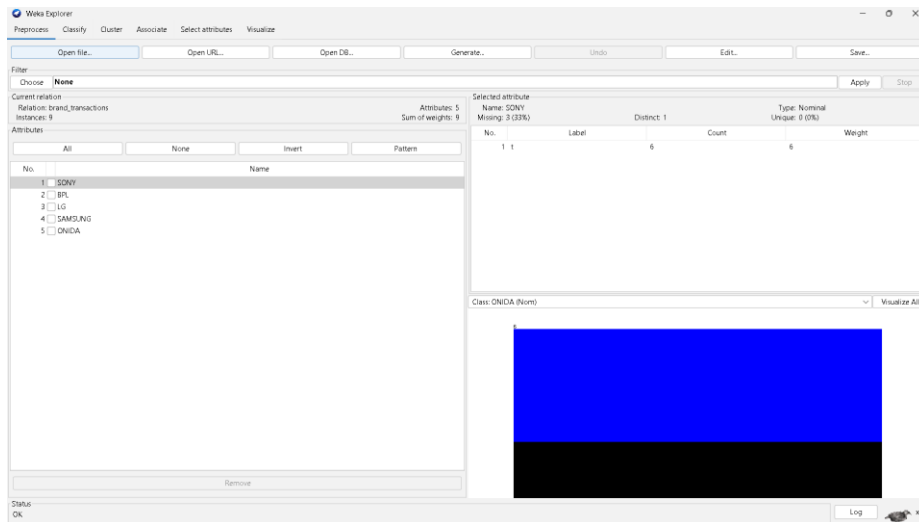
10.

Aim

To create an ARFF dataset for electronic brand transactions, implement the Apriori and FP-Growth association rule mining algorithms in WEKA with minimum support count $\text{min_sup} = 2$ and minimum confidence = 50%, compare the generated rules, and identify unique rules produced by each algorithm.

Algorithm (WEKA Association Mining Procedure)

1. **Dataset Representation:** Encode the 9 transactions with binary attributes for each unique item (SONY, BPL, LG, SAMSUNG, ONIDA) using t for present and ? for absent.
2. **Parameters Conversion:**
 - Total Transactions (N) = 9
 - Minimum Support Count = 2 $\Rightarrow \text{min_support} = \frac{2}{9} \approx 0.222$ (22.2%)
 - Minimum Confidence = 50% = 0.50
3. **Execute Apriori Algorithm:**
 - Open WEKA Explorer and load the ARFF file.
 - Go to the **Associate** tab and choose `weka.associations.Apriori`.
 - Set `lowerBoundMinSupport` to **0.22**, `minMetric` to **0.50**, and click **Start**.
4. **Execute FP-Growth Algorithm:**
 - In the **Associate** tab, choose `weka.associations.FPGrowth`.
 - Set `lowerBoundMinSupport` to **0.22**, `minMetric` to **0.50**, and click **Start**.
5. **Compare & Extract Rules:** Record the total rules from both algorithms, cross-check matching association rules, and highlight any differences in generation order or unique rule representations.



11.Aim

To perform data normalization on a batsman's strike-rate dataset (100,70,60,90,90) using four methods: Min-Max Normalization, Z-Score Normalization (Standard Deviation), Z-Score Normalization (Mean Absolute Deviation), and Decimal Scaling Normalization.

Step-by-Step Mathematical Calculations

Given values: $X = [100, 70, 60, 90, 90]$ ($n = 5$)

- **Statistical Parameters:**

- Min = 60, Max = 100, Range = $100 - 60 = 40$

- Mean (μ) = $\frac{100+70+60+90+90}{5} = \frac{410}{5} = \mathbf{82}$

- **Sample Standard Deviation (σ or s):**

$$\begin{aligned} s &= \sqrt{\frac{\sum (x_i - \mu)^2}{n - 1}} = \sqrt{\frac{(18)^2 + (-12)^2 + (-22)^2 + (8)^2 + (8)^2}{4}} \\ &= \sqrt{\frac{324 + 144 + 484 + 64 + 64}{4}} = \sqrt{\frac{1080}{4}} = \sqrt{270} \approx \mathbf{16.43} \end{aligned}$$

- **Mean Absolute Deviation (MAD):**

$$\begin{aligned} \text{MAD} &= \frac{\sum |x_i - \mu|}{n} = \frac{|18| + |-12| + |-22| + |8| + |8|}{5} = \frac{18 + 12 + 22 + 8 + 8}{5} = \frac{68}{5} \\ &= \mathbf{13.6} \end{aligned}$$

1. Min-Max Normalization (new_min = 0, new_max = 1)

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A = \frac{v - 60}{40}$$

- For 100: $\frac{100-60}{40} = \mathbf{1.0}$

- For 70: $\frac{70-60}{40} = \mathbf{0.25}$

- For 60: $\frac{60-60}{40} = \mathbf{0.0}$

- For 90: $\frac{90-60}{40} = \mathbf{0.75}$

- For 90: $\frac{90-60}{40} = \mathbf{0.75}$

2. Z-Score Normalization (Standard Deviation)

$$v' = \frac{v - \mu}{\sigma} = \frac{v - 82}{16.43}$$

- For 100: $\frac{100-82}{16.43} = \frac{18}{16.43} \approx +\mathbf{1.095}$
- For 70: $\frac{70-82}{16.43} = \frac{-12}{16.43} \approx -\mathbf{0.730}$
- For 60: $\frac{60-82}{16.43} = \frac{-22}{16.43} \approx -\mathbf{1.339}$
- For 90: $\frac{90-82}{16.43} = \frac{8}{16.43} \approx +\mathbf{0.487}$
- For 90: $\frac{90-82}{16.43} = \frac{8}{16.43} \approx +\mathbf{0.487}$

3. Z-Score Normalization (Mean Absolute Deviation)

$$v' = \frac{v - \mu}{\text{MAD}} = \frac{v - 82}{13.6}$$

- For 100: $\frac{100-82}{13.6} = \frac{18}{13.6} \approx +\mathbf{1.324}$
- For 70: $\frac{70-82}{13.6} = \frac{-12}{13.6} \approx -\mathbf{0.882}$
- For 60: $\frac{60-82}{13.6} = \frac{-22}{13.6} \approx -\mathbf{1.618}$
- For 90: $\frac{90-82}{13.6} = \frac{8}{13.6} \approx +\mathbf{0.588}$
- For 90: $\frac{90-82}{13.6} = \frac{8}{13.6} \approx +\mathbf{0.588}$

4. Decimal Scaling Normalization

$$v' = \frac{v}{10^j} \quad \left(\text{where } j = 3 \text{ since } \max(|v|) = 100 < 10^3 = 1000 \Rightarrow v' = \frac{v}{1000} \right)$$

- For 100: $\frac{100}{1000} = \mathbf{0.10}$
- For 70: $\frac{70}{1000} = \mathbf{0.07}$
- For 60: $\frac{60}{1000} = \mathbf{0.06}$
- For 90: $\frac{90}{1000} = \mathbf{0.09}$
- For 90: $\frac{90}{1000} = \mathbf{0.09}$

(Note: If normalized to the maximum two-digit boundary $10^2 = 100$, dividing by 100 gives 1.00,0.70,0.60,0.90,0.90).

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

Post Assistant Project (None)

Source

Console Background Jobs

```
> # R461 - ~/>
> # Input data
> x <- c(100, 70, 60, 90, 90)
>
> # (a) Min-Max Normalization
> min_max_norm <- (x - min(x)) / (max(x) - min(x))
>
> # (b) Z-score (Standard Deviation)
> z_score_sd <- (x - mean(x)) / sd(x)
>
> # (c) Z-score (Mean Absolute Deviation)
> mad_val <- mean(abs(x - mean(x)))
> z_score_mad <- (x - mean(x)) / mad_val
>
> # (d) Decimal Scaling Normalization
> j <- ceiling(log10(max(abs(x)) + 1)) # j = 3
> decimal_scaling <- x / (10^j)
>
> # Results Data Frame
> results <- data.frame(
+   Original = x,
+   Min_Max = round(min_max_norm, 4),
+   Z_Score_SD = round(z_score_sd, 4),
+   Z_Score_MAD = round(z_score_mad, 4),
+   Decimal_Scaling = round(decimal_scaling, 4)
+ )
>
> print(results)
  Original Min_Max Z_Score_SD Z_Score_MAD Decimal_Scaling
1    100    1.00    1.0954    1.3235         0.10
2     70    0.25   -0.7303   -0.8824         0.07
3     60    0.00   -1.3389   -1.6176         0.06
4     90    0.75    0.4869    0.5882         0.09
5     90    0.75    0.4869    0.5882         0.09
> |
```

Environment History Connections Tutorial

R - Global Environment

Data

results 5 obs. of 5 variables

Values

decimal_scaling	num [1:5]	0.1 0.07 0.06 0.09 0.09
j		3
mad_val		13.6
min_max_norm	num [1:5]	1 0.25 0 0.75 0.75
x	num [1:5]	100 70 60 90 90
z_score_mad	num [1:5]	1.324 -0.882 -1.618 0.588 0.588
z_score_sd	num [1:5]	1.095 -0.73 -1.339 0.487 0.487

Files Plots Packages Help Viewer Presentation

Zoom Export

12.Aim

To calculate the mean, variance, and standard deviation for the AvgSpeed and TotalTime attributes of 9 randomly selected cars.

Step-by-Step Mathematical Calculations

Given sample size: $n = 9$

- **Dataset:**

- AvgSpeed (X): [78,81,82,74,83,82,77,80,70]

- TotalTime (Y): [39,37,36,42,35,36,40,38,46]

1. Calculations for AvgSpeed (X)

- **Mean ($\bar{X}$):**

$$\bar{X} = \frac{\sum X_i}{n} = \frac{78 + 81 + 82 + 74 + 83 + 82 + 77 + 80 + 70}{9} = \frac{707}{9} \approx \mathbf{78.56 \text{ kph}}$$

- **Squared Deviations $\sum(X_i - \bar{X})^2$:**

- $(78 - 78.56)^2 = (-0.56)^2 = 0.3136$
 - $(81 - 78.56)^2 = (2.44)^2 = 5.9536$
 - $(82 - 78.56)^2 = (3.44)^2 = 11.8336$
 - $(74 - 78.56)^2 = (-4.56)^2 = 20.7936$
 - $(83 - 78.56)^2 = (4.44)^2 = 19.7136$
 - $(82 - 78.56)^2 = (3.44)^2 = 11.8336$
 - $(77 - 78.56)^2 = (-1.56)^2 = 2.4336$
 - $(80 - 78.56)^2 = (1.44)^2 = 2.0736$
 - $(70 - 78.56)^2 = (-8.56)^2 = 73.2736$
 - $\sum(X_i - \bar{X})^2 \approx 148.22$

- **Sample Variance (s_X^2):**

$$s_X^2 = \frac{\sum(X_i - \bar{X})^2}{n - 1} = \frac{148.22}{8} = \mathbf{18.53 \text{ (kph)}^2}$$

- **Sample Standard Deviation (s_X):**

$$s_X = \sqrt{s_X^2} = \sqrt{18.53} = \mathbf{4.30 \text{ kph}}$$

2. Calculations for TotalTime (Y)

- **Mean ($\bar{Y}$):**

$$\bar{Y} = \frac{\sum Y_i}{n} = \frac{39 + 37 + 36 + 42 + 35 + 36 + 40 + 38 + 46}{9} = \frac{349}{9} \approx \mathbf{38.78 \text{ mins}}$$

- **Squared Deviations $\sum(Y_i - \bar{Y})^2$:**

- $(39 - 38.78)^2 = (0.22)^2 = 0.0484$
- $(37 - 38.78)^2 = (-1.78)^2 = 3.1684$
- $(36 - 38.78)^2 = (-2.78)^2 = 7.7284$
- $(42 - 38.78)^2 = (3.22)^2 = 10.3684$
- $(35 - 38.78)^2 = (-3.78)^2 = 14.2884$
- $(36 - 38.78)^2 = (-2.78)^2 = 7.7284$
- $(40 - 38.78)^2 = (1.22)^2 = 1.4884$
- $(38 - 38.78)^2 = (-0.78)^2 = 0.6084$
- $(46 - 38.78)^2 = (7.22)^2 = 52.1284$
- $\sum(Y_i - \bar{Y})^2 \approx 97.56$

- **Sample Variance (s_Y^2):**

$$s_Y^2 = \frac{\sum(Y_i - \bar{Y})^2}{n - 1} = \frac{97.56}{8} = \mathbf{12.19 \text{ mins}^2}$$

- **Sample Standard Deviation (s_Y):**

$$s_Y = \sqrt{s_Y^2} = \sqrt{12.19} = \mathbf{3.49 \text{ mins}}$$

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function Addins

Post Assistant Project (None)

Source

Console Background Jobs

```
> # Input data
> avg_speed <- c(78, 81, 82, 74, 83, 82, 77, 80, 70)
> total_time <- c(39, 37, 36, 42, 35, 36, 40, 38, 46)
>
> # Calculations
> mean_speed <- mean(avg_speed)
> var_speed <- var(avg_speed)
> sd_speed <- sd(avg_speed)
>
> mean_time <- mean(total_time)
> var_time <- var(total_time)
> sd_time <- sd(total_time)
>
> # Summary Output
> results <- data.frame(
+   Variable = c("AvgSpeed (kph)", "TotalTime (mins)"),
+   Mean = round(c(mean_speed, mean_time), 2),
+   Variance = round(c(var_speed, var_time), 2),
+   Standard_Deviation = round(c(sd_speed, sd_time), 2)
+ )
>
> print(results)
```

	Variable	Mean	Variance	Standard_Deviation
1	AvgSpeed (kph)	78.56	18.53	4.30
2	TotalTime (mins)	38.78	12.19	3.49

Environment History Connections Tutorial

R - Global Environment

Data

results 2 obs. of 4 variables

Values

avg_speed	num [1:9]	78 81 82 74 83 82 77 80 70
mean_speed		78.5555555555556
mean_time		38.7777777777778
sd_speed		4.30439052338165
sd_time		3.49205447329283
total_time	num [1:9]	39 37 36 42 35 36 40 38 46
var_speed		18.5277777777778
var_time		12.1944444444444

Files Plots Packages Help Viewer Presentation

Zoom Export

13.Aim

To find all frequent itemsets from the transactional database using the Apriori and FP-Growth algorithms, compare their algorithmic efficiency, and extract all strong association rules matching the metarule:

$$\forall x \in \text{transaction}, \text{buys}(X, \text{item}_1) \wedge \text{buys}(X, \text{item}_2) \Rightarrow \text{buys}(X, \text{item}_3)$$

Step-by-Step Mining Execution

Given $N = 5$ transactions:

- $T100: \{M, O, N, K, E, Y\}$
- $T200: \{D, O, N, K, E, Y\}$
- $T300: \{M, A, K, E\}$
- $T400: \{M, U, C, K, Y\}$
- $T500: \{C, O, O, K, I, E\}$

(Assume minimum support threshold $\text{min_sup} = 60\% \Rightarrow \text{min_sup_count} = 3$ transactions, and minimum confidence $\text{min_conf} = 75\%$).

1. Frequent Itemset Generation

- **Item Support Counts (1-Itemsets):**
 - $K: 5$ ($T100, T200, T300, T400, T500$)
 - $E: 4$ ($T100, T200, T300, T500$)
 - $M: 3$ ($T100, T300, T400$)
 - $O: 3$ ($T100, T200, T500$)
 - $Y: 3$ ($T100, T200, T400$)
 - $N: 2$ ($T100, T200$ — *pruned*, < 3)
 - $C: 2$ ($T400, T500$ — *pruned*, < 3)
 - $D, A, U, I: 1$ each (*pruned*, < 3)
- **Frequent 1-Itemsets (L_1):**
 - $\{K\}: 5, \{E\}: 4, \{M\}: 3, \{O\}: 3, \{Y\}: 3$
- **Frequent 2-Itemsets (L_2):**
 - $\{K, E\}: 4$ ($T100, T200, T300, T500$)
 - $\{K, M\}: 3$ ($T100, T300, T400$)

- $\{K, O\}$: 3 (T100, T200, T500)
- $\{K, Y\}$: 3 (T100, T200, T400)
- $\{E, O\}$: 3 (T100, T200, T500)
- $\{E, M\}$: 2 (*pruned*)
- $\{O, Y\}$: 2 (*pruned*)
- **Frequent 3-Itemsets (L_3):**
 - Candidate generation from L_2 : $\{K, E, O\}$
 - Count for $\{K, E, O\} = 3$ (T100, T200, T500)
 - **Frequent 3-Itemset:** $\{K, E, O\}$ (Support = 3 / 60%)

2. Efficiency Comparison: Apriori vs. FP-Growth

Feature	Apriori Algorithm	FP-Growth Algorithm
Generation Strategy	"Generate-and-Test" candidate approach	Pattern fragment growth (divide-and-conquer)
Candidate Sets	Generates large sets of candidate k -itemsets (C_k)	Zero candidate itemsets generated
Database Passes	k database scans (scans data for each level)	Only 2 database scans
Data Structure	Array / Hash Tree	Compact FP-Tree
Efficiency	Lower efficiency, high I/O overhead	Highly scalable, significantly faster runtime

3. Strong Association Rules Matching the Metarule

The metarule requires 2 antecedent items and 1 consequent item:

$$\text{buys}(X, \text{item}_1) \wedge \text{buys}(X, \text{item}_2) \Rightarrow \text{buys}(X, \text{item}_3)$$

Derived from the frequent 3-itemset $\{K, E, O\}$ (support = 3 / 60%):

1. **Rule 1:** $\text{buys}(X, K) \wedge \text{buys}(X, E) \Rightarrow \text{buys}(X, O)$

$$\text{Confidence} = \frac{\sup(\{K, E, O\})}{\sup(\{K, E\})} = \frac{3}{4} = \mathbf{75\%}$$

(Meets $\text{min_conf} \geq 75\% \Rightarrow \mathbf{Strong Rule}$)

2. **Rule 2:** $\text{buys}(X, K) \wedge \text{buys}(X, O) \Rightarrow \text{buys}(X, E)$

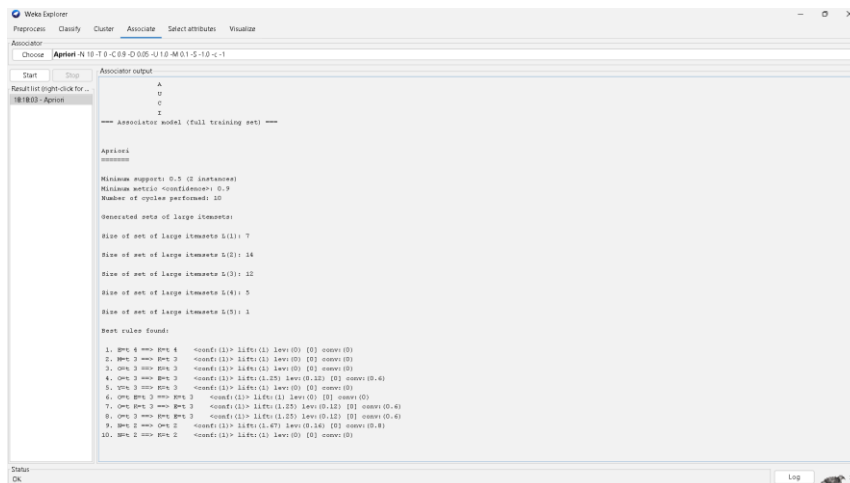
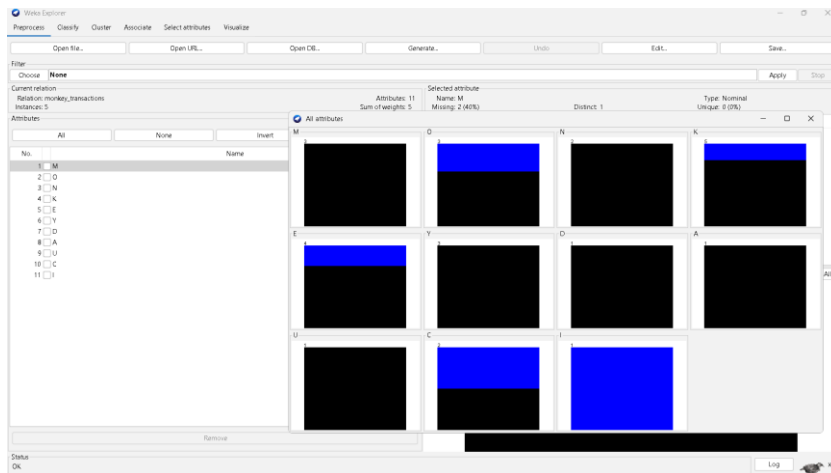
$$\text{Confidence} = \frac{\sup(\{K, E, O\})}{\sup(\{K, O\})} = \frac{3}{3} = \mathbf{100\%}$$

(Meets $\text{min_conf} \geq 75\% \Rightarrow \mathbf{Strong Rule}$)

3. **Rule 3:** $\text{buys}(X, E) \wedge \text{buys}(X, O) \Rightarrow \text{buys}(X, K)$

$$\text{Confidence} = \frac{\sup(\{K, E, O\})}{\sup(\{E, O\})} = \frac{3}{3} = \mathbf{100\%}$$

(Meets $\text{min_conf} \geq 75\% \Rightarrow \mathbf{Strong Rule}$)



Weka Explorer

Preprocess Classify Cluster **Associate** Select attributes Visualize

Associator

Choose **FPGrowth** -P 2 -I 1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1

Start Stop

Result list (right-click for ...)

181803 - Apriori

181837 - FPGrowth

Associator output

```
=== Run information ===

Scheme:      weka.associations.FPGrowth -P 2 -I 1 -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1
Relation:    monkey_transactions
Instances:    5
Attributes:   11
              M
              O
              N
              K
              B
              Y
              D
              A
              U
              C
              I

=== Associator model (full training set) ===

FPGrowth found 100 rules (displaying top 10)

1. [M=>]: 4 ==> [K=>]: 4 <conf(1)> lift(1) lev: (0) conv: (0)
2. [O=>]: 3 ==> [K=>]: 3 <conf(1)> lift(1) lev: (0) conv: (0)
3. [O=>]: 3 ==> [K=>]: 3 <conf(1)> lift(1) lev: (0) conv: (0)
4. [M=>]: 3 ==> [K=>]: 3 <conf(1)> lift(1) lev: (0) conv: (0)
5. [N=>]: 2 ==> [K=>]: 2 <conf(1)> lift(1) lev: (0) conv: (0)
6. [O=>]: 2 ==> [K=>]: 2 <conf(1)> lift(1) lev: (0) conv: (0)
7. [O=>]: 3 ==> [B=>]: 3 <conf(1)> lift(1.25) lev: (0.12) conv: (0.6)
8. [M=>]: 2 ==> [B=>]: 2 <conf(1)> lift(1.25) lev: (0.08) conv: (0.4)
9. [N=>]: 2 ==> [U=>]: 2 <conf(1)> lift(1.67) lev: (0.16) conv: (0.8)
10. [N=>]: 2 ==> [O=>]: 2 <conf(1)> lift(1.67) lev: (0.16) conv: (0.8)
```

Status
OK

Log  x 0